M04 PROTECTED PRACTICE REVIEW

Open only after a genuine saved attempt

Compare grain/cardinality/validation -> repair -> close -> changed SQL.

SA1 review - INNER JOIN and LEFT JOIN

1. What a strong attempt should demonstrate

State result grain correctly; INNER/LEFT behavior matches the business rule; unmatched count is explicit; no many-side table is treated as one-side without a uniqueness check.

2. Expected reasoning / worked pattern

Join orders to customers on customer_id. Then LEFT JOIN orders to return_events and verify that orders with no return remain visible with NULL return columns.

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

Unexpected missing rows -> check whether INNER JOIN should be LEFT JOIN. Unexpected extra rows -> inspect key uniqueness and one-to-many relationships before changing filters.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

SA2 review - Cardinality, row multiplication and join validation

1. What a strong attempt should demonstrate

Counts before/after are saved; expected multiplication is explained; right-side uniqueness is checked when assumed; at least one monetary total reconciles at the correct grain.

2. Expected reasoning / worked pattern

Join orders to order_items. Freight repeats for every item. Compare SUM(freight_amount) from orders alone with the naive post-join sum, then repair the analysis by aggregating item sales to one row/order before joining.

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

If totals inflate, do not add DISTINCT blindly. Identify the measure grain, aggregate the many side or compute the measure before the multiplication, then reconcile again.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

SA3 review - String functions for analyst cleaning

1. What a strong attempt should demonstrate

Raw and cleaned values remain comparable; only justified presentation changes are made; no UPDATE is used; one potentially ambiguous change is left for investigation.

2. Expected reasoning / worked pattern

Return raw customer_name/city beside TRIM(customer_name) and UPPER(TRIM(city)). Add a display label with CONCAT.

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

If cleaning changes business meaning, stop and keep the raw value. If spaces/case still differ, inspect hidden spaces and function order before applying more aggressive logic.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

SA4 review - Dates and time analysis

1. What a strong attempt should demonstrate

Elapsed days are correct on a manual sample; NULL ship dates are not treated as zero-day shipping; month results sort chronologically; one monthly count reconciles.

2. Expected reasoning / worked pattern

For completed orders, calculate DATEDIFF(ship_date, order_date), YEAR(order_date), MONTH(order_date), and a month_start expression.

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

If date functions return NULL/errors, verify data type and missing ship_date before changing formulas. If months sort incorrectly, use a true month_start/year-month key.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

SA5 review - Subqueries

1. What a strong attempt should demonstrate

Inner result shape matches how the outer query uses it; thresholds are independently queryable; result contains only rows/groups that satisfy the comparison.

2. Expected reasoning / worked pattern

Return order_items above average line_sales using a scalar subquery. Then compute customers above average customer sales with an aggregated inner result.

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

Subquery returns more than one row where one scalar is expected -> inspect the inner grain. Nested query feels unreadable -> rewrite with a CTE and compare clarity.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

SA6 review - CTEs and readable multi-step logic

1. What a strong attempt should demonstrate

Each CTE has a stated grain; stage counts are plausible; aggregate totals reconcile to the source; final output uses the intended stage rather than raw many-side rows.

2. Expected reasoning / worked pattern

Build order_sales as one row/order, validate count/total, then customer_sales as one row/customer and return the top customers.

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

If a later CTE has surprising rows, run the previous stage alone. If a CTE name hides its grain, rename it to make the row unit obvious.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

SA7 review - Window functions and PARTITION BY

1. What a strong attempt should demonstrate

Window query preserves the intended detail row count; partition totals match a separate GROUP BY check; ordering is deterministic where sequence matters.

2. Expected reasoning / worked pattern

On one row/order, add customer order count and customer freight total with COUNT/SUM OVER(PARTITION BY customer_id).

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

If row count collapses, you probably used GROUP BY instead of a window. If running totals are unstable on ties, add a deterministic tie-breaker to ORDER BY.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

SA8 review - Ranking, latest-row logic and LAG/LEAD

1. What a strong attempt should demonstrate

Latest/second-latest logic has deterministic tie-breaker; row count is one/customer where expected; monthly LAG aligns to chronological month_start; percentage change handles NULL/zero safely.

2. Expected reasoning / worked pattern

Rank orders per customer by order_date DESC, order_id DESC and keep rn=1. Build monthly sales and use LAG(monthly_sales) for prior-month comparison.

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

Unexpected duplicate latest rows -> ROW_NUMBER/tie-breaker issue. Wrong previous month -> verify month sequence and missing-month behavior before changing arithmetic.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

SA9 review - UNION, views and performance awareness

1. What a strong attempt should demonstrate

SELECT lists are type/column compatible; UNION ALL keeps meaningful duplicates; UNION use is justified rather than automatic; no server-administration task is performed.

2. Expected reasoning / worked pattern

Stack two compatible customer_id result sets with UNION ALL, then UNION, and identify a customer that appears twice only with UNION ALL.

3. Compare architecture, not text

Equivalent correct SQL is allowed. Compare business question, source/result grain, key/cardinality assumptions, row-count behavior and reconciliation before comparing exact syntax.

4. Likely repair

UNION column-count/type error -> align SELECT lists. Missing repeated business facts -> check whether UNION accidentally removed duplicates that should remain.

5. Fresh retry rule

Close this review. Use the lesson's changed transfer task or changed data/question. Type from a blank tab, validate independently, and append a new evidence row. Only the fresh retry can upgrade mastery.

M-SQLA project review - defend, do not copy

1. Expected architecture

Source grains -> key/cardinality proof -> one-row/order sales stage -> safe enrichment -> customer/product/month stages -> window/rank results -> return summary at one row/order -> reconciliations -> findings/limitations.

2. Critical reconciliations

Raw SUM(order_items.line_sales) must equal the validated order-level sales-stage total. Raw SUM(orders.freight_amount) must equal the corrected order-grain freight total after analysis. Count joins before/after and prove every assumed one-side key.

3. If it failed

Inflated money -> SA2. Join/unmatched issue -> SA1. Date/month issue -> SA4. Nested/stage grain issue -> SA5/SA6. Detail/window/rank/time-change issue -> SA7/SA8. Cohort stacking issue -> SA9.

4. Fresh retry

Close this review. Change one project question/cohort or rebuild the weak stage on a fresh query. Rerun downstream reconciliations before Gate A.